```
$ uv run life.py --ship
```

AN AI DEVELOPER CURRICULUM

From Zero to Shipped

22 weeks. 5 phases. Real apps, real users — and you can explain every line.

`Python` `FastAPI` `Supabase` `Railway` `Claude` `Stripe`

Your visual roadmap — keep this open alongside the sessions. `github.com/rysulliv/claude-coding-ta`

HOW THIS WORKS

The Deal

You build real apps with an AI mentor (Claude Code) that teaches the concept before the code, checks you followed, and quizzes you every session.

First you learn to understand and read code fluently; writing from scratch ramps up phase by phase — until you can build solo.

Every phase ends with something **deployed and used by a real person**.

> **“It works” is never the finish line.**
>
> **“I can explain why it works” is.**

Build real apps

A live URL in week 2. A database app your family uses by week 6.

Concepts before code

Every idea is taught — with its failure mode — before the code that uses it.

Read, explain, predict

≥2 checkpoints per session: read it back, predict the output, spot the bug.

Quiz every session

5–7 questions on what you just built. Misses come back until they stick.

BEFORE YOUR FIRST SESSION · NOTHING NEEDED BEFOREHAND · PART 1 OF 2

Day 0 — Accounts & Installs

New words, quickly: **GitHub** is where code lives online. A **repo** is a project folder with its full history. A **template** lets you stamp out your own personal copy of one — which is exactly what you'll do.

1	**Get a Claude subscription**	claude.ai → sign up → choose Pro (or Max). This one account powers your AI mentor.
2	**Create a GitHub account**	github.com/signup → free account. Username is public — pick one you'd show an employer (like firstname-lastname). Verify your email.
3	**Make YOUR copy of the course**	Go to `github.com/rysulliv/claude-coding-ta` → click the green **Use this template → Create a new repository** button. Name it something like my-coding-journey. (No template button? Click Fork — same result.)
4	**Install VS Code**	code.visualstudio.com → download → install with all the defaults. This is the editor everything happens in.
5	**Install Git**	git-scm.com/downloads → install, defaults are fine (Windows: Next through every screen). Then close and reopen VS Code so it notices Git.

That's every download and account. Python, databases, and the rest get installed WITH the mentor — as lessons.

BEFORE YOUR FIRST SESSION · PART 2 OF 2 · ~30–45 MIN TOTAL

Day 0 — Connect & Start

6	**Clone your repo in VS Code**	On your repo page, green “< > Code” button → copy the HTTPS link. In VS Code: Ctrl/Cmd+Shift+P → type “Git: Clone” → paste → pick a folder → Open. If a browser asks, click Authorize.
7	**Install the Claude Code extension**	Extensions view (Ctrl/Cmd+Shift+X) → search “Claude Code” → install the one published by Anthropic.
8	**Sign in**	Open the Claude panel (spark icon in the sidebar) → sign in with your Claude account in the browser window that opens.
9	**Sanity check**	Ask: “Can you see the CLAUDE.md and progress folder?” A yes means your mentor is loaded.
10	**Leave auto-accept OFF**	You review every change Claude proposes — always. Reading diffs is how you stay the author of your own code.
11	**Start**	Say: “Let's start the curriculum.” Session 0.1 opens with a full tour of everything you just set up.

Felt like a lot of clicking? Session 0.1 re-walks every step so you understand what you just did. Stuck? Ask the mentor itself, or see the README's troubleshooting section.

THE STACK — WHAT REAL STARTUPS USE, AT LEARNER PRICES

Your Toolbox

Python + uv

The language of AI work; uv runs it all

FastAPI

Your web server, with auto API docs

Supabase

Real Postgres + auth + storage, free tier

Railway

git push → live URL on the internet

Git + GitHub

Every change tracked, from day one

curl + Bruno

Test your API endpoints like a pro

Claude API

The AI inside your apps

Stripe

Real payments when you launch

Total cost: ~$10–25/month. Nothing here is a learning toy you'll throw away later.

THE ROAD — 5 PHASES, EACH ENDS WITH SOMETHING SHIPPED

22 Weeks, End to End

Wk 1–2	Wk 3–6	Wk 7–10	Wk 11–16	Wk 17–18	Wk 19–22+
0	1	2	3	4	5
Foundations	**Database App**	**Adding AI**	**API & Users**	**Go Deeper**	**Launch**
First live URL	*Family uses it*	*AI feature live*	*Closed beta*	*Solo rebuild*	*First dollar*

TO FINISH A PHASE, ALL THREE:

- **Deployed & used** — ≥1 real person actually using it
- **Phase exam ≥ 80%** — cumulative, no notes, no AI
- **Solo rebuild** — rebuild a piece from scratch, alone

The calendar is a suggestion. The gates are not — the curriculum adapts to you.

2–3 HOURS, EVERY TIME — THE MENTOR ENFORCES IT

Every Session, Same Loop

1. Plan

After a quick catch-up, you state the goal and sketch the approach.

2. Build

Concept first, then code. Claude explains; checkpoints prove you followed.

3. Walkthrough

Trace the whole code path. You narrate at least half.

4. Quiz

5–7 questions on today's concepts. Not optional.

5. Commit & Log

You explain the diff, write the commit message; it's pushed to GitHub.

Every ~4th session is a checkpoint: cumulative quiz, one exercise with zero AI help, and an “explain it to a non-programmer” write-up.

THE TWO MAGIC PHRASES — USE THEM EVERY DAY

Ending Your Day, Picking It Back Up

WHEN YOU'RE DONE FOR THE DAY, SAY:

“I need to stop here for today”

The mentor then runs the close ritual:

wraps the quiz, walks the commit gate, writes the handoff note, and pushes everything to GitHub.

Leaving suddenly? At least say “I have to go right now” so the handoff note gets written. Never just close the window.

NEXT TIME, OPEN VS CODE AND SAY:

“continue”

That's it. The mentor reads its notes and greets you with:

exactly where you stopped, the state of any half-finished code, and a couple of warm-up questions.

It feels like one continuous mentor — because the notes never sleep.

Why this works: the progress/ folder IS the mentor's memory — your journal, decisions, struggles, and mastery map live there as files in your repo. Each chat session is new; the files carry everything across. It only knows what got written down — so always end properly.

0

PHASE 0 · WEEKS 1–2

Foundations & First Deploy

Demystify the whole pipeline before going deep on any part of it — a live URL within 10 days.

WHAT YOU'LL BUILD

- Your workshop: VS Code + Claude Code, tuned and understood
- A Python CLI toy (Magic 8-Ball)
- A one-page FastAPI app, deployed on Railway

WHAT YOU'LL LEARN

- The terminal, git & GitHub
- Python basics: variables → functions
- Frontend vs. backend — who does what
- What a server actually is; reading logs & tracebacks
- Your first guided bug drill

SHIP IT: a live URL your family opens on their phones

1

PHASE 1 · WEEKS 3–6

Real App, Real Database

A CRUD web app on Postgres that a family member uses every week.

PICK ONE (OR INVENT YOUR OWN)

- Family Recipe Box
- Chore & Allowance Tracker
- Team Stats Tracker
- Gift Vault (secret claiming)

WHAT YOU'LL LEARN

- Schema design & SQL by hand
- psql + the Supabase dashboard
- FastAPI forms, Pydantic, HTMX
- JOINs and thinking in sets
- Production debugging: code, data, or config?

SHIP IT: an app with a database that real people put real data into

2

PHASE 2 · WEEKS 7–10

Adding AI

Your app gains genuinely useful AI features — and you understand tokens, prompts, and tools, not magic.

AI FEATURES IN YOUR APP

- Paste any recipe → AI structures it into the database
- Natural-language logging, parsed to rows
- “Chat with your app's data” via tool use
- Your first JSON API endpoint

WHAT YOU'LL LEARN

- The Messages API, tokens, context
- Prompting deliberately + A/B testing
- Structured JSON outputs & validation
- Streaming, tool use, agent loops
- Cost math, rate limits, prompt injection

SHIP IT: an AI feature your family actually uses (and you know what it costs)

3

PHASE 3 · WEEKS 11–16

API, Users & Security

A second, bigger app for people beyond family — built like a professional team would.

A PUBLIC-READY APP

- AI Study Buddy / Trip Planner / Hobby Coach…
- A versioned REST API (/api/v1)
- Full accounts: email + username, verify, reset, delete — optional “Sign in with Google”
- A Bruno test collection, kept green

WHAT YOU'LL LEARN

- REST design & status-code vocabulary
- JWTs, RLS, authn vs authz · attack your own API
- Race conditions, transactions, idempotency
- Caching & performance
- pytest, migrations, CORS, rate limits

SHIP IT: a closed beta with 5–10 friends creating accounts

4

PHASE 4 · WEEKS 17–18

Understanding What You Built

Two weeks deliberately going DOWN the stack, using your own apps as the case study. Never skippable.

THE DEEP DIVE

- Trace one request: DNS → TLS → your process
- Add an index to your slowest query — measure it
- Read the source of a library you use
- Unannounced bug drills begin

WHAT YOU'LL LEARN

- HTTP on the wire, status codes deeply
- What async actually does
- EXPLAIN, indexes, transactions
- Reading code you didn't write

SHIP IT: the graduation drill — rebuild a Phase 1 slice solo, zero AI, one sitting

5

PHASE 5 · WEEKS 19–22+

Monetization & Launch

The Phase 3 app goes public with a payment path. Even $1 of revenue changes how you think.

GOING PUBLIC

- Pricing & a free-vs-paid gate
- Stripe checkout + webhooks
- A landing page and a real domain
- Analytics, error alerts, uptime checks

WHAT YOU'LL LEARN

- Webhooks, signatures, idempotency
- Never trust the client about payments
- Activation & retention basics
- Handling strangers' feedback
- Terms, privacy & data responsibility

SHIP IT: a public launch in one real community — first stranger, first dollar

DEBUGGING IS A GRADED SKILL, NOT AN EMERGENCY

Bug Drills: Planted On Purpose

Throughout the curriculum, Claude deliberately plants realistic bugs. Your independence ramps up in four stages:

PHASES 0–1
Guided

Drill announced. Claude walks the full method out loud: error → hypothesis → test → fix → verify.

PHASE 2
Hinted

You drive the hunt. Claude answers direct questions only — hints on request, and they're counted.

PHASE 3
Solo

“There are two bugs in what we built. Find them.” Timed. Zero help. Graded.

PHASE 4+
Unannounced

Sometimes the code just misbehaves — like real life. Always revealed as a drill afterward.

Every drill ends the same way: name the bug's archetype, say how you'd prevent that whole class, and add the log line or test that would've caught it in 10 seconds.

QUIZZES, MEMORY & ADAPTATION

How the Score Is Kept

EVERY QUIZ MIXES

30% **Recall**

“What does a foreign key enforce?”

40% **Read code**

Your OWN code from today: “what does this print?”

30% **Predict**

“What happens if the user submits twice?”

Pass: 70% (80% on phase exams). Missed concepts come back in 1, 3, 7, then 14 days until you've beaten them.

The mentor remembers

Journal, decisions, struggles, and a mastery map — carried across every chat session.

Concepts recycle

Anything you miss goes into a review queue and reappears until it's retired.

The curriculum adapts

Crushing it? Material compresses (after a placement quiz). Struggling? Extra sessions appear — no judgment, no calendar guilt.

The gates never move

Deployed & used, exam ≥80%, solo rebuild. The path flexes; the bar doesn't.

THE ANTI-VIBE-CODING RULES

Rules of Engagement

- No commit until you explain the diff — and you write the message.
- Claude stops every ~30 lines to explain and check you followed.
- Every session: ≥2 checkpoints — read it back, predict the output, or spot the bug.
- Quizzes aren't skippable. Missed concepts return until beaten.
- One exercise per week with zero AI help.
- Auto-accept stays OFF — you review every change, always.

$ *You'll go slower than pure vibe coding — and in six months you'll be the one who actually knows how to build.*